

11

Running the simulators

This chapter describes how to run the Eclipse simulators:

- [Run an Eclipse simulation](#)
- [File handling in Eclipse](#)
- [Flux boundary conditions](#)
- [Licenses](#)
- [Parallel option](#)
- [Restarts](#)
- [Units](#)
- [Compatibility between Eclipse 100 specialist extensions](#)

Run an Eclipse simulation

There are several ways to run an Eclipse simulation:

- Using Petrel, you have access to a full suite of model building and checking modules. Having constructed your model you create a simulation case object. To do this, use the `Define simulation case` dialog box to select the simulator (Eclipse 100 for example) and define the inputs. You can then export the simulation case and optionally use the `Keyword editor` to edit the dataset before running the simulation. After running the simulation, you can use Petrel's visualization tools to analyze the results. If you already have datasets and results, you can import them into Petrel. Using Petrel's optimization and uncertainty capabilities, you can run several simulations and vary key parameters within set limits to determine the best production strategy for example.
- The simulator is installed with the `Simulation Launcher` and you can use this to select a simulator and dataset and then run a simulation. The launcher provides access to the Eclipse pre and post-processors which you can use to construct your dataset and visualize your results. For finer control, you can use advanced options to control the way in which the simulation runs. For more information, see the *Simulation Launcher User Guide* which you can get from the launcher's help menu.
- As a final option, you can run a simulation from a Windows or Linux command line. For information on the command options, see the *ECLRUN User Guide*.

File handling in Eclipse

x	Eclipse 100
x	Eclipse 300

A number of files may be produced by Eclipse for each simulation run. These are:

File	Purpose	Contents
PRINT file	Main printer output	RPT . . . keywords
DEBUG file	Debug output	DEBUG keyword
MSG file	Message output (XML format)	MSGFILE keyword
GRID file	Grid geometry file	NOGGF and GRIDFILE keyword)
SAVE file	Non-recurrent data for fast restarts	SAVE keyword)
INITIAL file	Initialization file for Petrel	INIT keyword
INSPEC file	Initial index file	INSPEC keyword in RUNSPEC section
RESTART file	Restart runs / grid graphics file	RPTSCHEd keyword
SMSPEC file	Specifications for summary file	SUMMARY section
SUMMARY file	Line graphics file	SUMMARY section
RSM file	Run summary file	SEPARATE keyword
RFT file	RFT and PLT graphics file	WRFTPLT keyword
RSSPEC file	Restart index file	RSSPEC keyword in the RUNSPEC section
EDESC file	Report step communication file for the Petrel Reservoir Geomechanics workflow.	
VDESC file	Stress step communication file for the Petrel Reservoir Geomechanics workflow.	
HMD file	Gradients file for SimOpt	HMDIMS in the RUNSPEC section). This keyword is used in the Eclipse 100 gradient option.
GRFT file	RFT gradients for SimOpt	GRADRFT in the SCHEDULE section). This keyword is used in the Eclipse 100 gradient option.

Table 11.1: Files produced by Eclipse for a simulation run

You can control the production of all of these files, except for the main printer output and VDESC files, with the keywords indicated.

The EDESC and VDESC files are produced by runs which use the Petrel Reservoir Geomechanics workflow. This workflow is activated by the VISAGE keyword in the RUNSPEC section. These small formatted files contain information on either all report dates (in the case of EDESC), or just the report dates at which stress step output has been generated (in the case of VDESC). It is processed by the Visage Coupler application in the Petrel Reservoir Geomechanics module which controls the Eclipse-Visage interaction.

The contents, and hence the size, of the printer output file are controlled using the RPTGRID, RPTPROPS, RPTREGS, RPTSCHEd, RPTSMRY, RPTSOL and RPTRST keywords.

In addition, you can choose whether input and output files are to be formatted or unformatted, (see the RUNSPEC section keywords [FMTIN](#) and [FMTOUT](#)), and whether RESTART and SUMMARY files are to be unified (see the RUNSPEC section keywords [UNIFIN](#) and [UNIFOUT](#)). You should be aware of which types of file it is appropriate to use on your system. Some guidelines are given here.

Formatted files are standard 80 character card image files, which may be straightforwardly transferred between different computers. All the files generated by the Eclipse programs can be converted from formatted to unformatted files or vice-versa using the utility program Convert.

In general, unformatted non-unified files should be used, giving efficient use of disk space and allowing you to examine output as the simulation progresses. However, it may be necessary to use formatted files if the simulation is performed on one type of computer but restarts or graphics are to be performed on another type of computer. Unified files should be used if the number of files being produced is a problem. Unified files also reduce the size of your catalog considerably (but not the amount of disk space used).

When reading an input file into Eclipse following any of the keywords [INCLUDE](#), [IMPORT](#), [RESTART](#) or [GDFILE](#), it is possible to set up aliases for pathname roots in order to facilitate transfer between different working directories and to avoid unnecessarily long pathnames.

A set of aliases for pathnames, each beginning with the character \$, can be defined with the RUNSPEC section keyword [PATHS](#).

Eclipse 100

File internal format

All Eclipse files consist of a series of data blocks each headed by a descriptor record. The block descriptor record comprises

- An 8-character keyword which identifies the data in the block.
- An integer (4 byte) containing the number of elements NEL in the block.
- A 4-character field specifying the type of the data in the block. Permitted values are:
 - 'INTE' for standard (4 byte) integers
 - 'REAL' for single precision (4 byte) floating point reals
 - 'LOGI' for standard (4 byte) logicals
 - 'DOUB' for double precision (8 byte) floating point reals
 - 'CHAR' for characters

Note: If the data type is 'CHAR', then NEL is the total number of CHARACTER*8 elements.

The contents of the block follows the descriptor (on a new record). This may be formatted or unformatted. If formatted, the form of output used may be read back by FORTRAN list input. If unformatted, each data block must be read back in the same block structure as it was written out.

The header/record structure enables files to be converted from formatted to unformatted form, or scanned for a specific item. For example, a RESTART file contains a record header with an identifier of 'PRESSURE', type 'REAL' and with the number of elements set to the number of active cells in the study.

In the formatted case the actual formats used for the data blocks are:

- Integers 6(1X,I11)

- Reals 4(1X,E16.8)
- Double Precisions 3(1X,D22.14)
- Logicals 25(1X,L2)
- Character strings 7(1X,A1,A8,A1)

The header line is written using a format (1X, A1, A8, A1, 1X, I11, 1X, A1, A4, A1). The A1 fields are used to output quotes so that the data may be read using list input.

'LOGIHEAD'													20 'LOGI'												
F	F	F	T	F	T	F	F	T	F	F	F	F	F	F	F	F	F	F	F	F	F	F			
'ZGRP'						1			'CHAR'																
'G'																									
'IWEL'						72			'INTE'																
1						1			3			3			1			1							
3						2			2			0			1			0							
1						2			0			2			0			-100							
0						0			0			0			1			1							
0						0			0			0			0			0							
0						7			0			0			0			0							
5						1			1			1			1			1							
1						4			4			0			1			0							
1						2			0			1			0			-100							
0						0			0			0			1			1							
-1						0			0			0			0			0							
0						7			0			0			0			0							
'PRESSURE'						15						'REAL'													
.39684106+004						.39684106+004						.39684106+004						.39684106+004							
.39684106+004						.39872349+004						.39872349+004						.39872349+004							
.39872349+004						.39872349+004						.40060627+004						.40060627+004							
.40060627+004						.40060627+004						.40060627+004													

For further information on formats of specific files, please contact the Support office.

Petrel files

Files written by Eclipse to be accessed by post-processing software, such as Petrel, comprise:

The `SUMMARY` and `SPEC` files. These are used to produce line graphs. Data is written to these files at every simulator timestep. Items to be written in this way are selected by keywords in the `SUMMARY` section of the input data set.

The GRID file	The GRID file, containing the locations of cell corner points. This file is always produced by Eclipse, unless a NOGGF keyword is placed in the GRID section to suppress it. The grid file contains information for plotting geometry displays of cell positions; it is also required for plotting solution data.
---------------	--

The solution or RESTART file	This file contains solution data (pressures and saturations for example) for all the cells at each report step.
------------------------------	---

Eclipse 100

The file content and report frequency may be controlled with the keyword `RPTRST`.

Eclipse 300

The solutions to be written must be chosen using the **RPTRST** keyword in the **SOLUTION** or **SCHEDULE** sections; otherwise the file is empty.

Flux boundary conditions

x	Eclipse 100
x	Eclipse 300

Flux boundary conditions enable runs to be performed on a small section of a field using boundary conditions established from a full field run. Flows across the boundary of the reduced field are written to a FLUX file at each mini timestep of the full field run.

The FLUX file is read during the reduced field run to generate the appropriate boundary conditions. Timesteps of the reduced field run may be quite different from those of the full field run.

A small amount of editing is required to convert a full field run that generates a FLUX file into a run, which runs one or two flux regions with the appropriate boundary flows being read from the FLUX file.

The following sections describe the use of the flux option in some simple cases.

Using flux boundary conditions

Single flux region

To produce the FLUX file:

1. Set NTFREG, the number of flux regions, to 1 in item 4 of keyword **REGDIMS** in the RUNSPEC section.

```
REGDIMS
-- NTFIP NMFIPR NTFREG NTFREG MHISTM NMHSTR
   1*      1*      1*      1      1*      1*
/
```

2. Enter the keyword **DUMPFLUX** in the GRID section, to indicate that this is the full field run and a flux file should be written.
3. Use the **FLUXNUM** keyword in the GRID section to establish the cells of the reduced run. For example, to establish flux boundary conditions for a reduced 2*2*2 run from a 4*4*3 full field run use:

```
DUMPFLUX

FLUXNUM
1 1 0 0
1 1 0 0
0 0 0 0
0 0 0 0
1 1 0 0
1 1 0 0
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
/
```

4. The flux file is called **ROOT.FLUX** (unformatted) or **ROOT.FFLUX** (formatted), where 'ROOT' is the root name of the run. The file will be formatted if keyword **FMTOUT** is specified in the RUNSPEC section.

Using the FLUX file in reduced runs

The reduced run is based on the original data set used for the full field (**DUMPFLUX**) run.

The reduced run should use the same grid dimensions as the full field run and employ the same flux region definition.

You nominate which flux regions are active in the reduced run. Eclipse then only simulates the reduced run within this active flux region. The support to this reduced flux region from the full field model is provided by a set of flows between flux regions which are supplied from the FLUX file dumped by the full field run.

To use the FLUX file in a reduced run:

Eclipse 100

1. Set NTFREG to 1 in item 4 of keyword **REGDIMS** in the RUNSPEC section.
2. Use the **USEFLUX** and **FLUXNUM** keywords in the GRID section, for example:

```

USEFLUX
  'ROOT'  /
FLUXNUM
  1 1 0 0
  1 1 0 0
  0 0 0 0
  0 0 0 0
  1 1 0 0
  1 1 0 0
  0 0 0 0
  0 0 0 0
  0 0 0 0
  0 0 0 0
  0 0 0 0
  /

```

3. If the `FLUX` file is formatted specify keyword `FMTIN` in the `RUNSPEC` section.
4. If user-specified non-neighbor connections are specified in the `GRID` section, the `NNC` keyword must be placed after the `USEFLUX` and `FLUXNUM` keywords.
5. The `FLUXNUM` array can be output to the `PRINT` file using the `FLUXNUM` mnemonic in the `RPTGRID` keyword.

Eclipse 100

In both the full field and reduced runs, the `FLUXNUM` keyword should be entered after the `DUMPFLUX` or `USEFLUX` keyword.

Multiple flux regions

Several flux regions may be defined and all relevant flows stored in the FLUX file. For example, 9 separate regions may be defined by setting NTFREG = 9 in the RUNSPEC section keyword REGDIMS and using keywords DUMPFLUX and FLUXNUM in the GRID section.

DUMPFLUX	FLUXNUM			
1	1	2	2	
1	1	2	2	
3	3	3	3	
4	4	4	4	
5	5	5	5	
6	7	8	8	
6	7	8	8	
6	7	8	8	
9	9	9	9	
9	9	9	9	
9	9	9	9	
9	9	9	9	
/				

Using the FLUX file in reduced runs

To perform a reduced run on a sector of the field defined by the 3 regions 2, 3 and 5:

Eclipse 100

1. Set NTFREG = 9 in item 4 of the RUNSPEC section keyword REGDIMS.
2. Use the `USEFLUX`, `FLUXNUM` and `FLUXREG` keywords in the GRID section. The `FLUXREG` keyword tells Eclipse which flux regions are active in the reduced run. The `FLUXNUM` keyword must be exactly the same as in the full field run. The `FLUXREG` and `FLUXNUM` keywords should follow the `USEFLUX` keyword.

```
USEFLUX
  'ROOT' /
FLUXREG
  2 3 5 /
FLUXNUM
  1 1 2 2
  1 1 2 2
  3 3 3 3
  4 4 4 4
  5 5 5 5
  6 7 8 8
  6 7 8 8
  6 7 8 8
  9 9 9 9
  9 9 9 9
  9 9 9 9
  9 9 9 9 /
```

Time extrapolation

By default, the simulation time in the reduced run cannot be longer than that in the full field model, as no boundary condition data is available for the period after the end of the full field simulation. However, if the simulation time of the reduced run is slightly longer than the field model run, the boundary conditions for that short period of time can be calculated by extrapolating the boundary data from the full field model. Eclipse will then freeze the last boundary condition stored in the FLUX file until the end of the run. To do this, item 2 of keyword `USEFLUX` should be set to 'YES'. You need to take care when selecting this, as extrapolation for a longer period time may result in unrealistic boundary conditions and lead to erratic results.

Treatment of wells

In the full field run, the oil, water, gas and reservoir fluid volume rates of each well are written to the FLUX file at each timestep in Eclipse 100. The oil, water, gas, voidage, wet gas and component well rates are written to the flux file at each timestep in Eclipse 300.

In the reduced run, the wells are sorted into two categories: those completed inside the flux boundary and those completed outside it. An error is flagged if a well has completions on both sides of the flux boundary. The wells outside the boundary are essentially ignored in the simulation, except that their flow rates are read from the FLUX file and added to the group and field totals. These wells are omitted from the well reports.

Because the wells outside the flux boundary do not exist as part of the simulation model, the ways in which they can influence the behavior of the wells inside the boundary are very limited. Essentially, if a group contains wells on both sides of the boundary (the FIELD will probably do so), then the only group control or constraint that can be applied to it is control of the group flow rate. The wells outside the boundary are treated as unavailable for group control, and their flows are read from the flux file and summed. The wells inside the boundary then make up the rest of the group's target rate. Other group constraints (for example economic limits, rate limits resulting in workovers, workover rig availability, artificial lift capacity, limits

on the number of open wells) should not be applied to groups containing wells on both sides of the boundary, as events happening outside the flux boundary are not taken into account. If the drilling queue contains wells both inside and outside the flux boundary, drilling events outside the boundary are only visible by the additional contribution of the new wells in the flux file at the end of the corresponding timestep of the full-field run. If the group falls short of its production target before the full contribution of the newly-drilled wells outside the boundary is seen, wells inside the flux boundary may be drilled prematurely in the `USEFLUX` run. It is best therefore to avoid using a drilling queue in these circumstances.

The well's reservoir fluid volume rate is by default calculated based on the average field hydrocarbon pressure.

Eclipse 100

The reservoir fluid volume rate calculated in the full field run will use the average hydrocarbon pressure over the whole reservoir and will be stored in the `FLUX` file. During the reduced run, the average pressure is read from the `FLUX` file at every timestep for the calculation of the reservoir volume rate in the well model.

Eclipse 300

The reservoir fluid volume rate calculated in the full field run will use the average hydrocarbon pressure over the whole reservoir while in the reduced run the average pressure is calculated based on the hydrocarbons inside the `FLUXNUM` region. However, you can specify that the well's reservoir fluid rate is calculated using the same average hydrocarbon pressure to avoid discrepancies between the full field and the reduced runs. To do this, define a `FIPNUM` region equivalent to the `FLUXNUM` region in the full field run and set item 13 of the `WELSPEDS` keyword to that `FIPNUM` region.

Drainage regions

Eclipse 100

Drainage regions are areas of the field that are drained separately from each other. These are written to the debug file if [item 25](#) in the `OPTIONS` keyword is set > 0 . Since flows are minimal between different drainage regions, this provides a convenient method of splitting the field into separate flux regions. If item 25 is set > 1 , then location of the main production well within each drainage region is identified by a negative region number. `FLUXNUM` treats negative numbers as if they were positive.

Aquifers

Aquifer data in the reduced run is processed in the usual way. Aquifers declared outside the flux region are ignored in the reduced run and the aquifer support is accounted for by the flux boundary condition generated from the `DUMPFLUX` run. For an aquifer that straddles a flux boundary, the aquifer support outside the flux region is accounted for by the flux boundary condition, but the flow of aquifer water into cells outside the flux region will be neglected; this will lead to a difference in the volumetric depletion of the aquifer for all aquifer types except a constant head/pressure or constant flux aquifer. This may cause a difference in the behavior of the sector between the reduced run and the full field run.

Note:

When using numerical aquifers with sector modeling, the inactive cells defining the aquifer through the `AQUNUM` keyword should not be included in the box defining the aquifer connections through `AQUCON`. Otherwise these connections will be eliminated during the run causing incompatibilities in the sector runs.

Restart runs

The reduced run can use flux information from a full field run that consists of a sequence of restarts. The `USEFLUX` keyword in the reduced run should specify the root name of the last Eclipse run in the full field

restart sequence. The FLUX files from all the restarts must exist and either be all formatted or all unformatted. Keyword `FMTIN` should be specified in the RUNSPEC section if the files are formatted.

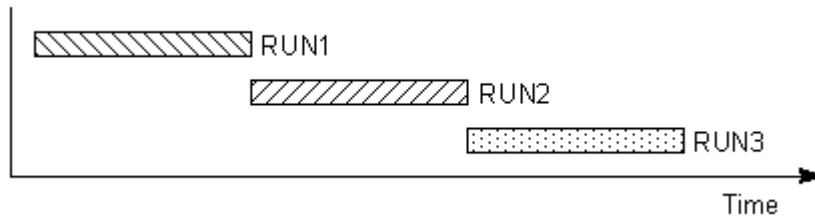


Figure 11.1. A sequence of restarts that together make a full field run

The reduced run `SMALL . DATA` includes the `USEFLUX` keyword as follows:

```
USEFLUX
'RUN3'
/
```

This run uses files `RUN1 . FLUX`, `RUN2 . FLUX` and `RUN3 . FLUX` to read flux information for the time sequence represented by all the rectangles. The number of restarts in the sequence is limited to 20.

It is possible to restart a reduced run from the corresponding full field run by specifying the full field root name in the `RESTART` keyword in the usual manner.

Note: The periods covered by the restarts in the sequence should not overlap.

Eclipse 300

Note: When doing a reduced run from a restart, the `GRID` file from the restart run must also be present in order for the simulation to initialize correctly.

Pressure boundary conditions (Eclipse 100)

Instead of using the flows of each phase from the full field run as boundary conditions on the reduced run, an alternative treatment using pressures and saturations is available (see the `FLUXTYPE` keyword). The pressures, saturations, R_s and R_v values in the 'halo' cells surrounding each flux region are stored in the FLUX file from the full field run. In the reduced run, Eclipse 100 calculates the flows across the flux boundary using the pressures and saturations in the halo cells surrounding the reduced flux region.

When the total flux across the boundary of the reduced region does not alter appreciably, the default flow boundary condition is adequate. This is the case, for example, when the flux option is used to examine the effect of changing the well data in the reduced region when the flux boundary is wholly an inflow boundary or wholly an outflow boundary and the distribution of fluids on the flux boundary does not alter appreciably in the reduced run. When more severe changes are made to the flow regime and fluid distribution, the pressure boundary condition should be used to allow the phases to redistribute adjacent to the boundary and allow the switch from inflow/outflow in the full field run to outflow/inflow in the reduced run. In this case, Eclipse 100 works out whether the flow is into or out of the reduced region by upwinding, which, in general, improves stability.

No-flow boundary conditions (Eclipse 100)

As an alternative to defined-flux or defined-pressure boundary conditions, a `USEFLUX` run may be performed with no-flow boundary conditions. The `USENOFLO` keyword should be entered in the `GRID`

section. In this case it is not necessary to provide a FLUX file. This option provides a simple way of simulating just a section of the reservoir in isolation (provided that no-flow boundary conditions are a reasonable approximation) without having to perform a DUMPFLUX run.

Local grid refinement and coarsening

Eclipse 100

Within Eclipse 100, refined grids and coarsenings must not be placed at flux boundaries in a DUMPFLUX run.

Eclipse 100

In a reduced flux USEFLUX run, local grids and coarsenings cannot straddle or be placed at boundaries of the reduced flux region (defined by FLUXREG). However, they can be placed within flux regions provided that there is at least one cell between the refined or coarsened cells and the flux boundary.

Eclipse 300

In a reduced flux USEFLUX run, local grids cannot straddle or be placed at boundaries of the reduced flux region (defined by FLUXREG). However, they can be placed within flux regions provided that there is at least one cell between the refined cells and the flux boundary. Coarsenings cannot straddle boundaries, but may be placed at them.

Refined grids or coarsenings outside the reduced field region are ignored in a reduced field run. Thus, as with wells, it is unnecessary to edit out local grid refinements or coarsenings in the inactive region of a reduced field run. It is not possible for individual LGRs within a single LGR amalgamation to belong to different flux regions. When running reduced flux USEFLUX runs in parallel, amalgamations may not be used outside of the USEFLUX region.

Reading Eclipse 100 flux files into Eclipse 300

Flux runs are usually run by Eclipse 300 and use a FLUX file generated by Eclipse 300. It is possible to read an Eclipse 100 FLUX file. An Eclipse 100 FLUX file contains flows in terms of stock tank oil and gas volumes, and you must then also enter XSTC and YSTC, the stock tank oil and gas compositions, in order that the program can generate component flows.

It is not possible to use an Eclipse 100 FLUX file to run a thermal reduced data nor it is possible to use it to run a model with the "Water components" option active.

Limitations to the use of flux boundary conditions with CEOR agents and tracers (Eclipse 100)

All CEOR agents and tracers are supported in reduced runs with pressure boundary conditions (FLUXTYPE 'PRESSURE') with some constraints on flow into the flux region as explained below. Support with flow boundary conditions (FLUXTYPE 'FLUX') is more limited. Temperature (TEMP) and is not supported in reduced runs of either type.

Eclipse outputs an error if unsupported quantities are used in a reduced run. The error can be suppressed by using item 232 of the OPTIONS keyword. This option may be useful when it is known that the flow of these quantities across flux boundaries does not occur.

Pressure boundary conditions are compatible with TRACER, POLYMER, SOLVENT, SURFACT, BRINE (used by itself) and BRINE when used together with POLYMER. If outflow of CEOR agents or tracers across sector boundaries is available in a reduced run but inflow across sector boundaries is not, you need to ensure that all the EOR/Tracer activity occurs within the sector.

The following table summarizes the compatibility of all of the options.

EOR agent or tracer	Outflow across pressure boundary	Inflow across pressure boundary	Outflow across flow boundary	Inflow across flow boundary
POLYMER	Y	Y	N	N
BRINE when used with POLYMER	Y	Y	N	N
BRINE	Y	Y	Y	Y
SURFACT	Y	Y	Y	N
SOLVENT	Y	Y	N	N
ALKALINE	Y	N	N	N
Passive tracers (TRACER) in oil, water, gas	Y	Y	Y	Y
Partitioned tracer PARTTRAC	Y	N	N	N
API	Y	Y	N	N
TEMP	N	N	N	N

Table 11.2: Support for CEOR agents and tracers in reduced runs.

Thermal option

Eclipse 300

The flux boundary option can be used for 2 different scenarios:

1. Full field model is a Thermal model, and the reduced model is also a Thermal model
2. Full field model is either an Eclipse 300 Black-oil model or a Compositional model, and the reduced run is a Thermal model.

For option 1, a thermal reduced run can be carried out from a thermal base run. In this case, the energy flows across the boundary are written out to the FLUX file in the full field run.

For option 2, a thermal reduced run can be carried out from a compositional or an Eclipse 300 black-oil base case. In this case the energy flows across the boundary are calculated in the reduced run.

The conductive term is defined as $C_{ei} = - \sum_n \Psi_{ni} \Lambda (T_i - T_n)$,

where

Ψ_{ni} is the heat transmissibility between cells n and i . This term is calculated as follows:

$$\Psi_{ni} = \alpha \times \gamma_{ni}$$

where γ_{ni} is the transmissibility between cells n and i taken from the base run, and

α is the heat transmissibility factor, defined using the THFLXTF keyword.

The Λ term is the saturation dependent transmissibility multiplier of the cell inside the active flux region.

The convective term is defined as $F_{ei} = \sum_n \gamma_n \left(\sum_p M_p^e dP_{pni} \right)$,

where

M_p^e is the energy generalized mobility, and is calculated as follows:

$$M_p^e = H_p \cdot k_{rp}(S_p) \cdot \frac{b_p}{\mu_p}.$$

Referring to the above form, the $k_{rp}(S_p) \cdot \frac{b_p}{\mu_p}$ part is taken from the base run, and the phase enthalpy H_p is calculated in the cell inside the active flux region during the reduced run.

To enable a thermal reduced run from a non-thermal base run, item 1 of the [FLUXOPTS](#) keyword should be used in the base run and set to YES. This allows Eclipse 300 to provide all the essential information to perform the reduced thermal simulation.

Water Components option

Eclipse 300

The flux boundary option can be used for two different scenarios when combined with the ["Water components"](#) option:

1. The full field model has the Water Component option active, and the reduced run also has the Water Components option active.
2. The full field model has the standard water model active (water properties input with [PVTW](#)), and the reduced run has the Water Components option active.

For option 1, it is required that the same number of water components ([COMPW](#)) is present in the full field model and in the reduced run.

For option 2, the water related properties read from the [FLUX](#) file are all assumed to be from the first water component defined in the reduced run and set to zero for any other water component. This set up allows simulating the injection of waterborne components in a given section of the reservoir without need to run the full field model. It is important that the properties of the first water component input in the reduced run match the properties of the water in the full field model as described in ["Converting PVTW data to water component data"](#). In addition, you should ensure that the newly injected fluids do not change the flow conditions on the flux boundary; this might be achieved by selecting a flux boundary large enough so that the newly injected fluids do not cross the flux boundary.

Using flux boundary conditions to achieve similar results to the full field model

The use of flux boundary conditions should give results close to the full field model if nothing is changed in the reduced run. In practice, there are changes in the reduced run that affect the boundary conditions. In such cases it is important to ensure that the changes made in the sector do not significantly affect the boundary conditions.

When selecting which boundary type to use, consider which boundary conditions should be kept unchanged when changes are made in the sector model. For instance, if well pressures must be matched a flow boundary should be used to preserve the flux across the boundary, and if rates need to be adjusted a pressure boundary should be used to preserve the pressure around the sector model. Note that some options are only available for either the pressure boundary or flow boundary conditions.

Reduced runs may be able to run choosing time steps larger than the full field simulation. If so, when comparing simulation results from the reduced run against the full field, small numerical differences may be observed in the grid blocks belonging to the flux boundary. These differences can be overlooked for as long as they remain at grid block level and do not alter the overall flows in and out of the flux boundary. If

a larger level of accuracy is required at the boundary grid blocks, this could be achieved by modifying the reduced run time stepping controls to match those of the full field.

Current limitations

- `DUMPFLUX` and `USEFLUX` cannot be used simultaneously in the same run.
- `SUMMARY` section keywords which depend upon information from outside of the active region in a `USEFLUX` case will not include that information; volume quantities such as fluid in place will not sum in volumes from inactive cells and flow quantities will not include flows where either cell is inactive.
- Gas Field Operations • The flux boundary option should not be used in conjunction with automatic DCQ reduction in the gas field operations model.
- Parallel Option • Using `DUMPFLUX`, the WARP linear solver cannot be used, instead the pre-2002A linear solver is activated.
- The nine-point schemes (`NINEPOIN` keyword) cannot be used when the flux boundary option is used.

Summary of keywords

RUNSPEC

- `REGDIMS` Item 4 specifies the maximum number of flux regions in the run.

GRID

Restriction	Keyword	Description
	<code>ADDREG</code>	Adds a constant to the specified array in a flux region.
	<code>COPY</code>	Copies data from one array to another.
	<code>DUMPFLUX</code>	Makes a full field run write a <code>FLUX</code> file.
	<code>EQUALREG</code>	Sets an array to a constant in a flux region.
	<code>FLUXNUM</code>	Identifies extent of each flux region.
	<code>FLUXREG</code>	Identifies which flux regions are active.
	<code>FLUXTYPE</code>	Specifies the type of flux boundary condition.
	<code>MULTIREG</code>	Multiplies an array by a constant in a given flux region.
Eclipse 100	<code>RPTGRID</code>	Controls on output from the <code>GRID</code> section: <code>FLUXNUM</code> outputs flux region numbers.
	<code>USEFLUX</code>	Specifies the <code>FLUX</code> file used for a reduced run.
Eclipse 100	<code>USENOFLO</code>	Allows a <code>USEFLUX</code> data set to be run without a <code>FLUX</code> file.
Eclipse 300	<code>XSTC</code>	Specifies oil composition at standard conditions.
Eclipse 300	<code>YSTC</code>	Specifies gas composition at standard conditions.
Eclipse 300	<code>THFLXTF</code>	Specifies the heat transmissibility or flow transmissibility ratio T

Options keywords

Restriction	Keyword	Description
Eclipse 100	OPTIONS	Activates special program options item 25: Drainage regions are written to the DEBUG file. item 65: Remove certain error messages about wells connecting in multiple flux regions, or for local grids adjacent to flux boundaries in a DUMPFLUX run. item 232: Removes the ban on EOR/tracer functionality with DUMPFLUX and USEFLUX runs. In this case, you must ensure that on EOR/tracer data is needed from outside the flux boundary.
Eclipse 300	OPTIONS3	Activates special program options item 69: Remove certain error messages about wells connecting in multiple flux regions in a DUMPFLUX run.
Eclipse 300	FLUXOPTS	Activates special program options Item 1: When set to YES, makes a full-field run output the necessary information for a thermal reduced run. Item 2: Provides a factor to improve convergence on non-thermal reduced models.

Licenses

x	Eclipse 100
x	Eclipse 300

To run Eclipse, and also to run any of the additional Eclipse options, you need to obtain an appropriate FLEX license.

The FLEX license checking procedure checks for the availability of an appropriate license for an Eclipse option when a keyword activating the feature is encountered. In most cases, the activating keyword is in the RUNSPEC section, so is encountered at the beginning of the run.

Activating keywords for some features, however, may not be encountered until the SCHEDULE section, and perhaps even part way through a run. For example, the Standard Network option is activated by the GRUPNET keyword, and the Gas Field Operations Model may not be activated until the SWINGFAC keyword is read. If an appropriate license is not available when a particular feature is activated, the run stops.

A DATACHECK license is available, which allows Eclipse to run a data set in NOSIM mode to check for data errors without taking up a full license.

Eclipse 100

To avoid the run stopping part way through, it is possible to reserve all the required licenses at the beginning of the run, using the LICENSES keyword in the RUNSPEC section to identify the features in the run for which licenses will be required.

Eclipse 300

The LICENSES is only needed in order to use a sim_compositional or sim_thermal license in place of an Eclipse-only thermal or compositional license.

Note: The LICENSES keyword is also used by the Eclipse launchers to work out which licenses are used, except for parallel licenses, in order to implement the license aware job scheduling for Eclipse 100 and Eclipse 300.

This chapter provides information about the licenses used by specific simulators. A description of the keywords required to invoke these features is also given.

There are a number of special cases which will be dealt with separately:

- OpenEclipse
- Parallel
- Block Parallel
- Multiple Realization (MR).

Licenses used by specific simulators

In most cases, the licenses being used by the simulators are quite straight forward to work out. A base eclipse license is checked out and, depending on other factors, additional licenses may also be used.

Primarily it is the keywords in the data set that determine which licenses are required. Standard usage is described below along with some exceptions and special cases.

Eclipse 100 uses a base eclipse license. A more complex example involving reservoir coupling is as follows:

It is possible to couple several Eclipse models together. See "Reservoir Coupling". If a data set is coupled to others, it will contain the SLAVES keyword. In this case, the total license requirement is the same as if

all the data sets had been run concurrently, with the additional requirement for a rescoupling license. For example, if the following data sets are coupled:

- A master data set which only requires an `eclipse` license.
- A slave which requires an `eclipse`, `lgr` and a `networks` license.
- Another slave that requires an `eclipse`, `lgr` and a `multisegwells` license.

The total license usage is 3x `eclipse`, 2x `lgr`, 1x `networks`, 1x `multisegwells` and 1x `rescoupling`.

Eclipse 300 typically requires a `compositional` (or `sim_compositional`) and an `eclipse` license. Again, there are several possibilities.

- If the Eclipse 300 data set contains the `THERMAL` keyword, then Eclipse 300 will only check out an `eclipse` and a `thermal` (or `sim_thermal`) license.
- If you have an `e300` license available it will be used instead of an `eclipse` and a `compositional` license. However, the `e300` license is no longer sold.
- If the data set contains the `BLACKOIL` keyword, you can run Eclipse 300 in black-oil mode with an `eclipse` license and there is no need for a `compositional` license. Similarly if the data set contains the `TEMP` keyword, you can run Eclipse 300 in black-oil mode with an `eclipse` license and there is no need for a `thermal` license.

Which keywords invoke which features?

The following table below shows the feature names, the keywords that invoke the functionality and the license key name. Where two license names appear, the simulator will attempt to check out the first one and if this fails it will attempt to check out the second.

License Feature Name	Keywords	License Key	E100	E300
Compositional	COMPS	<code>compositional</code> or <code>sim_compositional</code>		x
Datacheck	NOSIM	<code>datacheck</code>	x	x
Eclipse SynSize	PEDIMS, GRADRESV	<code>eclipse4d</code>	x	
Well Bore Friction (MAINTENANCE MODE)	FRICITION	<code>friction</code> <code>multisegwells</code>	x	x
Lagrangian Tracer Solver	LAGTRAC	<code>fs_streamtrace</code> <code>frontsim</code>		x
Gas Field Operations	GASFIELD, GASPERIO, GASYEAR, SWINGFAC	<code>gasfield</code>	x	x
Gas Lift Optimization	GLIFTOPT, LIFTOPT, WLIFTOPT	<code>gaslift</code>	x	x
Local Grid Refinement / Coarsening	LGR, CARFIN, RADFIN, AUTOREF, COARSEN, AUTOCOAR	<code>lgr</code>	x	x

License Feature Name	Keywords	License Key	E100	E300
Multiple Realization	MULTREAL is generated by a pre-processor	multiple_realisation parallel	x	x
Multisegment wells	WSEGDIMS, WSEGSOLV	multisegwells	x	x
Networks	NETWORK, GNETINJE, GRUPNET	networks	x	x
OpenEclipse Runtime License	Called by controller	openeclipse	x	x
Parallel Eclipse	PARALLEL	parallel multiple_realisation	x	x
Reservoir Coupling	DUMPCUPL, GRUPMAST, GRUPLAV, RCMASTS, SLAVES, USECUPL	rescoupling	x	
Reservoir Optimization	RESOPT	resopt		x
Thermal	THERMAL	thermal or sim_thermal		x
Unencoded Gradients	FMTHMD, GDIMS, HMDIMS, RPTHMD, UNCODHMD	unencodedhmd	x	x
Water Services	RIVRDIMS, SAMG and summary vectors reporting hydraulic heads.	waterservices	x	

Table 11.3: Feature names, keywords that invoke the functionality and the license key name

Special Cases

OpenEclipse

OpenEclipse enables Eclipse 100 and Eclipse 300 to be controlled by, and communicate interactively with, other applications. Although Eclipse is primarily used in a batch standalone mode controlled solely by the contents of the input data file, there are many situations in which it would be advantageous to have Eclipse controlled by another software application.

For instance, there may be a requirement to have Eclipse tightly coupled to a surface gathering system model, or a specialized production optimization application. It could also be useful to couple Eclipse to an interactive controller, to allow you to view the current status of the simulation and make well management decisions during the course of the run. In addition to the licenses that Eclipse usually checks out, the simulator will also request an openeclipse license.

Parallel

The parallel option allows the simulation of a single data set to be distributed across a number of processors. This allows large simulations to be carried out in a shorter time than would normally be the case.

Both Eclipse 100 and Eclipse 300 are optimized to provide the shortest time to solution, and consequently tailor the number of domains, and hence the linear solver, to the number of available processors.

This has a number of consequences. Firstly, the results obtained on different numbers of processors will agree to engineering accuracy, not to machine accuracy. Secondly, the scalability is suboptimal, as the linear solver becomes less efficient as the number of processors increases.

Running Eclipse in parallel requires the same set of licenses as running in serial, plus a number of `parallel` or `multiple_realisation` licenses, which can be used interchangeably. The total number of these licenses is equal to the first argument of the `PARALLEL` keyword.

“parallel license” is used in the following sections to mean `parallel` or `multiple_realisation` licenses.

The number of parallel licenses required is independent of how the model is run. As an example, say an Eclipse model is split into 8 domains, which could be run in a number of configurations. For instance, 1 computer with 8 processors, or 2 computers running 4 processes each, or 8 computers running 1 process each. For all these cases, the model is split into 8 domains, so 8 parallel licenses would be required. If the model was split into 16 domains, then 16 parallel licenses would be required.

Block Parallel

The Block Parallel license is activated by the `BPARA` keyword. This is an alternative license option for parallel simulations with up to 32 parallel domains. In such cases, a single Block Parallel license can be used instead of the (up to) 32 parallel licenses that would normally be required. To use this option the `PARALLEL` keyword must also be present, with item 1 (the number of domains) set to a value in the range 2 to 32. If more than 32 domains are requested the simulation will stop with an error message. The Block Parallel license feature is not compatible with the Multiple Realization option.

Multiple Realization

Multiple Realization (MR) licensing was implemented as a way to lower the cost of running multiple similar versions of the same model.

Firstly, you generate MR data sets using programs such as Petrel or MEPO. The data set differs from a standard data set in that it includes the `MULTREAL` keyword.

Following the `MULTREAL` keyword is a UID (Unique Identifier), which is a long string of numbers and a word (‘yes’ or ‘no’). All MR data sets generated from the same base case at the same time will have the same UID. The word, Yes or No, tells Eclipse whether to continue if MR or parallel licenses are not available. This is however not yet implemented in the generating applications.

The parallel and multiple realization licenses used to be separate. From the 2011.1 release onwards, they are used interchangeably.

When the first of a group with a single UID starts running, the standard set of licenses that would be required for that job is checked out. In addition, an MR/parallel license is checked out as well. When the second and subsequent jobs from the same group start, they each only check out an MR/parallel license. Even if the first job finishes, the standard set of licenses are kept checked out until all running jobs from that group finish.

Parallel and reservoir coupling are excluded from this licensing scheme, and if any data set includes these features, they will be checked out as normal. This means that if you want to run Nmr realizations of a parallel case with Nprocs domains, you will require the following total number of MR/parallel licenses: Nmr (one for each MR case) + Nmr * Nprocs (for the overall number of parallel licenses needed).

Example

If you want to run 20 realizations of an initial Eclipse model that requires a LGR license:

Without MR licenses you would require 20 Eclipse Black Oil + 20 LGR licenses.

With MR licenses you will need 1 Eclipse Black Oil + 1 LGR license + 20 Eclipse Parallel/MR licenses

If the same model requires 8 way parallel, you will require 1 Eclipse Black Oil + 1 LGR license + 20 Eclipse Parallel/MR licenses + 8*20 Eclipse Parallel/MR licenses.

The example in figure 11.2 shows 6 MR jobs running through a system that can handle 3 jobs at a time.

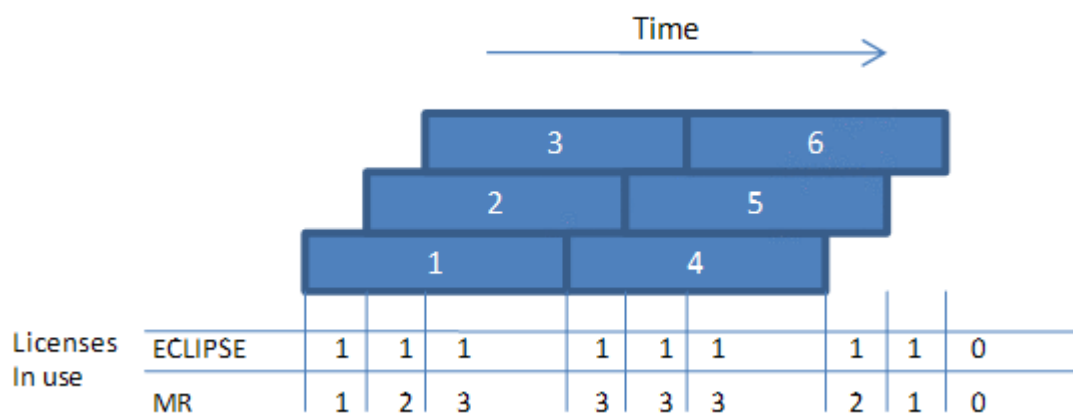


Figure 11.2. Example shows 6 MR jobs running through a system that can handle 3 jobs at a time

License compatibility

		Multiple realization	
		Single	n multiple scenarios
Parallel	1 (serial)		n parallel
	m ways, no BPARA	m parallel	n + n*m parallel
	m ways, with BPARA (m<= 32)	1 block parallel	Banned

Table 11.4: License compatibility and requirements for parallel, block parallel and multiple realization

Lagrangian Tracer Solver

The Lagrangian tracer solver feature uses a FrontSim streamline (fs_streamtrace) license in the first instance. However, if no fs_streamtrace license is available then a full frontsim license will be taken instead. A Petrel locked FrontSim license will not activate this feature.

Listing of available licenses

The list of available licenses is not output by default. This output can be enabled by setting the `LISTLICENSES` keyword to `TRUE` in the configuration file.

2014.1 license retirements

This release of Eclipse allows a number of options to be used without the need for a special license where previous releases had required one. These options are as follows:

- CO₂ Storage
- Coal Bed Methane Model
- Environmental Tracers
- Flux Boundary Conditions
- Foam Model
- Gas Calorific Value Control
- Polymer Flood Model
- Gi Pseudo-Compositional Model
- Solvent Model
- Surfactant Model

2015.1 license retirements

Eclipse 300 geomechanics has been disabled in this release of Eclipse. The following licenses are now redundant and do not invoke any features:

- Geomechanics
- Proppant flowback in a fracture
- Sand Management Advisor

Parallel option

x	Eclipse 100
x	Eclipse 300

The Parallel option allows the simulation of a single data set to be distributed across a number of processors. This allows large simulations to be carried out in a shorter time than would normally be the case with the standard simulators. Both Eclipse 100 and Eclipse 300 are optimized to provide the shortest time to solution, and consequently tailor the number of domains and hence the linear solver to the number of available processors. This has a number of consequences. Firstly, the results obtained on different numbers of processors will agree to engineering accuracy, not to machine accuracy. Secondly, the scalability is poorer than one might expect, as the linear solver becomes less efficient as we go to larger numbers of processors.

You can change the number of processors at restarts depending on the resources available, using flexible restarts.

The Parallel option is designed to be easy to use; you only need one additional keyword to run a data set in parallel mode. You may then use additional keywords to tune the data set for optimal performance.

Activating a parallel run

A parallel run of Eclipse 100 or Eclipse 300 is activated by using the `PARALLEL` keyword in the RUNSPEC section. Suppose the following is used:

```
PARALLEL
16 /
```

When the Eclipse run is submitted, 16 copies of the executable will be spawned. Each of the 16 tasks is then running a copy of the simulator, dedicated to its section of the reservoir. A message passing system is used to pass boundary flow and well information between the tasks and keep them synchronized.

One of the tasks performs file input and output; it is in other respects the same as the remaining tasks, dealing with its own section of the reservoir.

In most respects, running Eclipse 100 and Eclipse 300 in parallel is the same as running in the standard serial mode. However, there will always be some features or combinations of features which are not available with the parallel option.

The Parallel option is intended for use on multi-core computers or clusters with high performance interconnects. Parallel runs must have dedicated resources, which means that the number of Eclipse processes (the number of domains + the number of serial runs) on any machine should not exceed the number of CPUs on that machine. If the machine is overloaded the performance degradation may be severe as most message passing systems assume a dedicated resource, and processes that are waiting for messages from other processes will often consume CPU time at the expense of those that are trying to do calculations.

Partitioning

Parallel versions Eclipse 100 and Eclipse 300 are fundamentally different in two respects. Firstly, the well model calculations of Eclipse 100 are not done in parallel. This has a minor impact on the scalability of Eclipse 100. The second and more important difference is in the approach used in the making the linear solver run in parallel. Eclipse 100 uses a modified nested factorization approach, which maintains full coupling across the grid at each linear iteration. Eclipse 300 uses a domain decomposition approach.

Eclipse 100 restricts you to a one-dimensional domain decomposition, but maintains full coupling of the solution across the reservoir at each linear iteration, at the expense of increasing the work required per

linear iteration by 1.3. We would however not expect the number of linear iterations to increase significantly as we increase the number of domains. Eclipse 100 parallel runs are restricted to three dimensional data sets.

Eclipse 300 allows a two-dimensional decomposition and requires no extra work per linear iteration, at the cost of breaking the coupling across the reservoir at each linear iteration. We would therefore expect the number of linear iterations to increase with the number of domains. (Note that each domain is coupled to its neighbor only at each linear iteration, but the reservoir is fully coupled at the level of each Newton iteration.)

In both Eclipse simulators a balance has to be struck between increasing the amount of work per linear iteration and increasing the number of linear iterations.

Eclipse 100

Eclipse 100 partitions the reservoir either in the x or the y direction, whichever is the outer solver direction. It automatically divides the reservoir into domains with approximately equal numbers of active cells.

The number of planes allocated to each domain can be modified using the [DOMAINS](#) keyword in the `GRID` section. Since each domain is processed by an individual processor, it is advantageous to have broadly similar numbers of active cells within each domain. The partitions of the reservoir and the solver directions can be output to the Print file by including the `DOMAINS` mnemonic in `RPTGRID`, or graphically using the domains output to the `INIT` file.

Options to improve performance

A number of options that may affect the performance of the code can be specified using the [PARAOPTS](#) keyword.

- The use of a 'tree' or 'hypercube' algorithm for global reductions such as sums and maxima. In several places within the parallel code quantities have to be summed across domains/processors. This can either be done by sending messages from each processor to one processor (in practice processor 1) where they can be added and the result broadcast back to the other processors. Under some circumstances this can lead to a communications bottleneck at processor 1.
- The use of the hypercube algorithm in which processor 2 sends its data to processor 1, and processor 4 sends its data to processor 3 which performs a sum or maximum operation on the available data and then forwards the results to processor 1. The disadvantage is that the algorithm has several levels to it, but it does reduce the number of message into a single processor.
- The [DOMAINS](#) keyword can be used to control the size of domains.

Efficiency and performance estimation

The theoretical efficiency of the parallel option needs to be broken down into three components:

- The modified linear solver algorithm may require more iterations than the purely serial solver (that is the algorithm may be less powerful). This effect will vary from model to model; however, experience to date suggests that the degradation in performance is small in most cases.
- The modified algorithm requires more work even when the number of linear iterations remains the same. The extra work is required to allow the majority of the code to be made to run in parallel. This increase is typically between 15% and 30% of the original run, for numbers of processors in the range 4 to 32. Two processors is a special case with typically no overhead, as no extra filling terms are required.

- The critical issue for a parallel algorithm is the fraction of the code that remains serial. It is critical because this is the fraction that cannot be reduced by increasing the number of processors; it is the limit on the possible speed-up of the run.

The work that does not run in parallel consists of the calculations not associated with the grid block solution (for example wells and I/O). In addition, the section of the linear solver required to communicate information between domains is not wholly parallel, in the sense that some processors must remain idle.

The proportion of the total CPU time spent in the serial sections of the code can be investigated using the Eclipse timing facility. If the mnemonic 'CPU' is specified in keyword [RPTSCHEd](#), Eclipse produces a timing report. This report is slightly different in the parallel case. Four extra mnemonics have been added:

Below GINV:

GAM is the tree elimination for the gamma calculation.

Below SRCH:

FORW is the parallel forward sweep through each domain

TREE is the tree elimination to communicate between domains

BACK is the parallel backward sweep through each domain.

An example is shown below:

```
=====
TIMING REPORTS AT CPU=    955 SECS - ELAPSED=    989 SECS
=====
ZZSS      1  0  0
              RR      1  0  0
SR         1  0  0
TS        20100100
              NX      52 99 99
              WC      231  0  0
              LMRS    231  1  1
                  LMRI  231  0  0
                  LMRO  231  0  0
              CONV    231  0  0
              LMJC    179  2  2
                  LMJI  179  0  0
                  LMJO  179  0  0
              LXX     179 78 78
                  WGIN  179  0  0
                  GINV  179 19 18
                      GAM  179  2  2
              WRES    179  0  0
              ITER    179 59 59
                      NORM 1720  3  3
                      SRCH 1541 41 41
                          FORW 1541 15 15
                          TREE 1541  2  2
                          BACK 1541 16 16
                      WADX 1541  0  0
                      ORTH 1541 12 12
                      UPDA 1541  3  4
                  WSOL  179  0  0
              XUIT    179  1  1
              LCMI    179 16 16
XW         20  0  0
          31 READING END

PARALLEL
16 'S' /
```

Eclipse 100 optionally allows you to run the parallel algorithm on a single processor by specifying 'S' after the number of processors in the parallel code.

If a model is run on a single processor, the theoretical speed-up can be estimated from the fractions under the `GAM` and `TREE` mnemonics, provided that the well solution and I/O sections of the code only consume a negligible fraction of the CPU time. Note that the `GAM` and `TREE` sections of code are not genuine serial sections but include the serial component. The following application of Amdahl's law can be thought of as a conservative measure of the theoretical speed-up.

$$S = GAM + TREE \quad \text{Eq. 11.1}$$

Serial fraction:

Theoretical parallel CPU time:

$$\left(\frac{1-S}{NP} + \frac{S}{NP-1} \log_2(NP) \right) CPU_{serial} \quad \text{Eq. 11.2}$$

NP is the number of processors (usually = number of domains)

where

The actual performance will be less than this theoretical value for a number of reasons, including:

- Load balancing, where the amount of work on each processor is not exactly equal.
- Overheads associated with the implementation on the parallel computer.

In the distributed memory case these include the latency and bandwidth limitations of the message passing system. In the shared memory case these include the startup cost of the parallel loops and the bottleneck of all processes accessing the memory at the same time.

Eclipse 300

In a parallel run the reservoir is partitioned areally into domains, each of which is assigned a processor. The default is to divide the reservoir into one-dimensional strips, along the reservoir's largest N_x or N_y direction. However, for large numbers of domains, a two-dimensional decomposition is desirable, to avoid thin strip domains. For example, with 16 processors, a 4 by 4 layout of domains can be realized by:

```
NPROCX
4 /
NPROCY
4 /
```

If the basic simulation grid is 120 by 120 by 10, this will imply that each processor treats a domain of dimension 30 by 30 by 10.

For reservoirs with a significant number of inactive cells, the default partitioning is not sufficient to load-balance the domains; that is ensure all domains have roughly the same number of active cells and therefore work. It is possible to control further the way in which cells are assigned to domains. By default, the x- or y- dimension of the grid is equally divided. However, this may be overridden using the `PSPLITX` and `PSPLITY` keywords in the `RUNSPEC` section. Suppose, in the case above, we also specified

```
PSPLITX
60 120 /
40 80 120 /
40 60 80 120 /
/
```

The division of cells in the x-direction for the 4-processor case would then be as shown below. Note that this keyword also contains partitions for the 2 and 3 processor cases.

Process	Cells
1	1 to 40
2	41 to 60
3	61 to 80
4	81 to 120

Table 11.5: The division of cells between processors using the PSPLITX keyword.

The keyword PSPLITX is analogous. The simulator will output good estimates of these keywords by using the PART mnemonic in the RPTSOL keyword. By performing an initial NOSIM run, then pasting the suggested PSPLITX and PSPLITX output in the RUNSPEC section, good load balancing can be achieved for all combinations of processors. The percentage active cell load balancing of the current run is reported to the PRT file. This is defined

$$\text{Active cell load balance \%} = \frac{\text{total active cells in reservoir}}{\max(\text{active cells per partition}) \times \text{number of procs}} \times 100$$

This figure represents the maximum parallel speed-up that can be achieved for a given run. For example, if the active cell load balance is 75% for a 4-processor job, then the maximum parallel speed-up that can be achieved is 3 (ignoring any super-linear speed-up that may be achieved by cache effects). The actual parallel speed-up achieved may be much lower than this. Consequently it is advisable to monitor this figure, especially if large parts of the reservoir are inactive. Eclipse 300 gives suggestions for better partitions if the PART mnemonic is put in the RPTSOL keyword in the SOLUTION section. The PRINT file contains suggestions on how the current partition may be improved. In addition the Debug file contains a list of all the possible combinations of PDIVX and PDIVY. This information can only be obtained once the GRID section is processed. A two-stage procedure is required to utilize the partition suggestions from Eclipse 300.

1. Run the simulator with default partitions with the PART mnemonic in the RPTSOL keyword (no SCHEDULE section is necessary).
2. If the active cell load balance is low, paste the relevant PDIVX and/or PDIVY output from the Debug file of the first run into the RUNSPEC section, and run again.

To examine the partition of a particular run, the PART mnemonic can be placed in the RPTRST keyword, and then plotted from the RESTART file using any standard package (for example, Petrel). This is useful to determine if any wells cross partition boundaries.

Note the WARP linear solver splits each parallel partition into a number of sub-grids, which are connected through NNCs; see "[The Eclipse 300 WARP linear solver](#)" for a detailed description.

Running LGRs in parallel (Eclipse 100)

Parallel domain decomposition runs can include local grids. These are defined in the normal way. By default, in Eclipse 100 each local grid is treated as a complete entity and is allocated to one of the slave processes. This will usually be the most efficient approach if there is a large number of local grids; if we have only a few LGRs, however, there will be a load imbalance. Each slave process can handle any number of local grids. Each LGR can be allocated manually to a specific process using the IHOST keyword. Alternatively, Eclipse 100 can allocate the LGRs automatically. While doing this Eclipse 100 attempts to balance the number of active refined cells.

In Eclipse 300 the refined cells inherit their host processor from the global grid. Consequently, LGRs are split across processors. If there is a large number of LGRs it can be difficult to achieve a good load balance.

Performance bottlenecks

There are several possible reasons for performance bottlenecks.

- A poor active cell load balance. This can only be addressed by re-partitioning the grid. Some user intervention is required to improve the partitioning of any grid. You can examine the partition graphically using the `RESTART` file by putting the `PART` mnemonic in the `RPTRST` keyword, and suggestions for better partitions can be acquired by putting the `PART` mnemonic in `RPTSOL`.
- If completions of a well are on a different processor to the well head (specified, for example, by the `COMPDAT` and `WELSPEC` keywords) then off-processor completions will result, and the 10th argument of `WELLDIMS` needs to be set to the maximum allowed. Do not set this argument to too high a number. Off-processor completions will result in a significant communication overhead, and will consequently degrade parallel performance. If possible, these should be avoided, either by moving the location of the well head or re-partitioning. If all the horizontally-completed wells lie in the I or J direction, then re-partitioning may be a straightforward operation: just partition in the I or J direction to avoid the off-processor completions. However, off-processor completions are very often unavoidable if horizontally completed wells are present.
- The parallel linear solver is a variant of nested factorization. The significant feature is that the serial and parallel algorithms are different. This results in some slight differences in results between the serial and parallel runs. More importantly, the parallel algorithm often requires more iterations to converge, and so more work is required by the parallel run which inevitably may result in some degradation of parallel performance. This can be monitored using the `NLINEARS` mnemonic in the `SUMMARY` section.
- Disk output is a solely sequential activity, and so all processes within a parallel job will wait while one process writes to disk. Excessive output will significantly degrade overall parallel performance.
- Numerical aquifers connect a group of cells, representing the aquifer, to the reservoir. If the group of cells (defined using `AQUNUM` keyword) or the connections from the aquifer to the reservoir (defined using `AQUCON` keyword) cross partition boundaries, then extra communication will result. This will degrade parallel performance. If such connections can be easily avoided it is advisable to do so.
- LGRs are allowed with the parallel option. In Eclipse 300 the partition of the LGRs is 'inherited' from the global grid. Any LGRs that lie across partition boundaries are automatically split. If many nested LGRs are present, then achieving a good active load balance can be problematic, given the constraints of the global partition. In addition, LGRs that are split across domain boundaries can have an adverse effect on the linear solver performance.

In Eclipse 100 LGRs are processed as on one processor and the data swapped between processors. The default distribution of LGRs tries to place approximately equal numbers of refined cells on each processor. However, mixing LGRs in place and locally timestepped LGRs can destroy this load balancing as these are processed in completely different ways. Also, locally timestepped LGRs do not all take the same number of linear or non-linear iterations, or local timesteps. Hence it is possible that the smallest LGR in terms of the number of refined active cells may be the slowest to perform a given global timestep. In addition, the most difficult local grid to solve may change during the run. All this can destroy any attempt to load-balance the LGR calculations.

- It is possible to connect any two cells of the grid through the `NNC` keyword. If the cells lie on different partitions, a noticeable communication overhead in conjunction with degraded linear solver performance may result.

Memory scaling

Eclipse 100 and Eclipse 300 allocate memory as needed using dynamic memory management. Therefore, the `MEMORY` keyword is not needed anymore.

The memory resources will reduce as the number of processors (as specified in the `PARALLEL` keyword) increases. This is a consequence of the individual domains becoming smaller as the number of processors increases. Thus less storage per domain is needed. How the memory requirements scale as the number of processors used for a particular job increases is very case-dependent, and is not linear. All the well and tabular data is stored simultaneously by all processes. Consequently, severely over-dimensioning the maximum number of wells or completions, for example, will waste memory on each process with a parallel job. This could become expensive as the number of processors used in a parallel job is increased.

Output (Eclipse 100)

When running the parallel option in Eclipse 100, output may appear in a different file location than in serial. For each processor, additional files are created to store the output from that processor. Additionally, terminal output for processors other than the master are written to a separate log file. All of these files have the same file types as the main output files but have the processor number appended after the file root and before the file type. No additional files are created for processor number 1, the master processor. An example of the files output in parallel is:

File type	Main output file	Output file for processor n
Print	BASE.PRT	BASE.n.PRT
Debug	BASE.DBG	BASE.n.DBG
Message	BASE.MSG	BASE.n.MSG
Log	Output to terminal	BASE.n.LOG

Table 11.6: Example of the files output in Eclipse 100 when using the parallel option.

At the end of a successful run, output files other than the master are deleted. The additional files are retained if either a bug or error occurs or if item 2 of the `PARAOPTS` keyword is set to 1. Output controlled by keywords beginning `RPT` may also not be output in parallel unless item 8 of the `PARAOPTS` keyword is set to 1.

Messages, comments, warnings, errors and bugs from all processors are included in the main output files in parallel. In the main output files, the number of the processor that produced a message is included in the message header. For example:

```

@--MESSAGE  ON PROCESSOR 1 AT TIME      0.0  DAYS    (1-JAN-2000) :
@           THE MEMORY REQUIRED TO PROCESS THE SOLUTION SECTION IS
@           1679 MBYTES  PROCESSOR    1
@           1329 MBYTES  PROCESSOR    2
@           1284 MBYTES  PROCESSOR    3
@           1261 MBYTES  PROCESSOR    4

```

If identical messages are repeated simultaneously across processors then the message will only be output once in the main output files. If a very large number of messages are output in short succession then the parallel message buffer may become full. If this happens, the buffer size will be increased automatically. If insufficient memory is available then the code will exit. This issue can be avoided by reducing the maximum number of messages output using the `MESSAGES` keyword

Summary of keywords

RUNSPEC

Restriction	Keyword	Description
Eclipse 300	NPROCX	Defines the number of processors to be used in the x-direction.
Eclipse 300	NPROCY	Defines the number of processors to be used in the y-direction.
	PARALLEL	Initializes the Parallel option.
Eclipse 300	PDIVX	Defines the partitioning in the x-direction.
Eclipse 300	PDIVY	Defines the partitioning in the y-direction.
Eclipse 300	PSPLITX	Specifies multiple domain decomposition in the x-direction.
Eclipse 300	PSPLITY	Specifies multiple domain decomposition in the y-direction.

GRID (Eclipse 100)

Eclipse 100 keyword	Description
DOMAINS	Defines the partitioning of the grid.
IHOST	Allocates LGRs to specific processes.
PARAOPTS	Permits some fine tuning of the simulator.
RPTGRID	Controls output from the GRID section. The DOMAINS argument outputs the domain split.

SOLUTION section (Eclipse 300)

Eclipse 300 keyword	Description
RPTRST	Controls solution output for flexible restarts. Argument PART outputs domain partitions
RPTSOL	Controls output from the SOLUTION section. Argument PART outputs domain partitions.

Restarts

x	Eclipse 100
x	Eclipse 300

There are two methods of restarting an Eclipse run:

- A fast restart, which reads the RUNSPEC, GRID, EDIT, PROPS and REGIONS section data from a SAVE file. The data is stored in a processed form, so that for example the transmissibilities do not have to be recalculated. The SAVE keyword must be used in RUNSPEC section of the original run to tell Eclipse to construct a SAVE file. An important restriction with fast restarts is that the SAVE file must have been produced by the current version of Eclipse.
- A flexible restart, which reads a complete data file. This is the slower method, as the data must be processed again (for example the transmissibilities are recalculated). However, it does allow more flexibility, as it is possible to change some of the data items from their values in the original run, and restarts from files produced by earlier versions of Eclipse are possible.

Both methods require the time-dependent data (for example grid block solution variables and the status of wells) to be stored at one or more times in the original simulation run. By default, the time-dependent data is stored in the RESTART file. Alternatively, the time-dependent restart data may be appended to the SAVE file, so that the RESTART file can be used purely for graphics output. See ["Requesting RESTART files"](#).

Fast restarts from a SAVE file

Eclipse 100

Restarting a run from a SAVE is usually quicker to initialize than a restart from a complete data file. This method can be used for simple restarts where only the SCHEDULE section data is to be modified.

The procedure is as follows:

1. In the base run, use the [SAVE](#) keyword to instruct Eclipse to create a SAVE file, and use the [RPTSCHEd](#) or [RPTRST](#) keywords to control the frequency with which the solution data is to be written to the RESTART file.
2. Set up the data file for the restart run, consisting of a [LOAD](#) keyword, a [RESTART](#) keyword, the SUMMARY and SCHEDULE section data.

For example,

```
LOAD
  BASE /
RESTART
  BASE 11 /
SUMMARY
INCLUDE
  SUMMARY.DAT /
SCHEDULE
...
...
```

Note: The LOAD keyword replaces the RUNSPEC, GRID, EDIT, PROPS and REGIONS sections, and opens the SOLUTION section where the RESTART keyword is expected. The SAVE file that it names must have been produced by the current version of Eclipse - fast restarts cannot be performed from SAVE files produced by earlier versions.

Eclipse 100

Note: In Eclipse 100, for a parallel fast restart run the `PARALLEL` keyword should not be present in the restart data file since the record is read from the `SAVE` file instead. However, a parallel fast restart run must be executed on the same number of processors as the base run.

Eclipse 100

Note: Fast restarts are not supported in with the `PARALLEL` option in Eclipse 100 runs containing local grid refinements

Eclipse 300

Note: The use of fast restarts is not compatible with the `PARALLEL` option in Eclipse 300.

- a. In the `SCHEDULE` section, unless keyword `SKIPREST` has been entered, delete the well and timestepping keywords up to and including the restart time at report number 11 in the example (but keep any VFP tables, as these are not passed over in the Restart file). If keyword `SKIPREST` is present, you need not delete these keywords as Eclipse automatically skips the unwanted keywords until the restart time has been reached in the Schedule data. Modify the Schedule data after the restart time as required.

3. Restarting from time-dependent data stored in the `SAVE` file:

- a. If the time-dependent restart data has been stored in the `SAVE` file, the `RESTART` keyword requires up to two extra items of data, for example:

```
RESTART
BASE 11 SAVE FORMATTED /
```

- b. The string `SAVE` indicates that the restart data is present in the `SAVE` file, and the string `FORMATTED` indicates that the `SAVE` file is formatted. By default the `SAVE` file is assumed to be unformatted.

Flexible restarts from a data file

A run can be restarted by supplying a complete data file, and taking the initial solution from the restart file. This method has the advantages of not requiring the `SAVE` file, and of allowing restarts from older versions of Eclipse. It also enables the restart to be run with modified data, for example the maximum number of wells can be increased (but not reduced). It is possible to restart a run with, for example, altered permeability data, but in general such changes are not recommended.

To do a restart from, say, the 11th report time of run `BASE`, the procedure is:

1. Think of a new file name root - say `R1` .
2. Copy the data file `BASE . DATA` into `R1 . DATA`.
3. Edit `R1 . DATA`
 - a. In the `SOLUTION` section, delete all equilibration or enumeration keywords, and any analytic aquifer keywords. Insert the `RESTART` keyword to specify the restart file and report number,

```
RESTART
BASE 11 /
```

- b. In the SCHEDULE section, unless keyword `SKIPREST` has been entered, delete the well and timestepping keywords up to and including the restart time at report number 11 in the example (but keep any VFP tables and thermal Temperature Cutback tables, as these are not passed over in the restart file). If keyword `SKIPREST` is present, you need not delete these keywords as Eclipse automatically skips the unwanted keywords until the restart time has been reached in the SCHEDULE data. Modify the schedule data after report 11 as required.

4. Submit the run with R1 as the root filename.

The restart run is slower to initialize than a restart from a SAVE file, because the geometry data has to be processed to calculate the pore volumes and transmissibilities for example.

It is possible also to do a flexible restart from time-dependent restart information stored in a SAVE file, even though the SAVE file has not been used to provide the non-recurrent data as it would in a fast restart. In this case the RESTART keyword needs up to two extra items of data, for example:

```
RESTART  
BASE 11 SAVE UNFORMATTED /
```

This keyword requests that the time dependent data for the restart is taken from the unformatted SAVE file generated by the base run.

Requesting RESTART files

By default, the restart data written at each report time will be stored in separate RESTART files. The files' suffix indicates the report number at which the data was written. Optionally, the time-dependent restart data written at each report can be stored in a single "Unified" RESTART file instead. If you wish to produce a Unified RESTART file, enter the keyword `UNIFOUT` in the RUNSPEC section. To enable Eclipse to read a Unified RESTART file, the keyword `UNIFIN` must be present in the RUNSPEC section - otherwise Eclipse will look for non-unified RESTART files.

By default, any RESTART files created are unformatted files. If you wish to write formatted RESTART files instead, enter the keyword `FMTOUT` in the RUNSPEC section. To enable Eclipse to read formatted RESTART files, the keyword `FMTIN` must be present in the RUNSPEC section - otherwise Eclipse looks for unformatted RESTART files. Files may be converted from formatted to unformatted and vice-versa by means of the utility program Convert. See "File Handling in Eclipse" for more details on file formats.

Eclipse 100

To create RESTART files in Eclipse 100, from which a later run can restart, the procedure is:

1. Set either the mnemonic BASIC in the `RPTRST` keyword > 0 , or set the mnemonic RESTART in the `RPTSCHED` keyword > 0 .
 - a. If either mnemonic is set to 1, Restart files are created at every report time, but deleted when a later restart is available. This facility is intended to provide a fail-safe restart. (Note that this facility is not available if unified Restart files are being output).
 - b. If the mnemonic = 2 (or = 1 also for unified files), Restart files are created at every report time until the switch is reset. None are deleted.
 - c. Giving the mnemonic BASIC in the `RPTRST` keyword a value > 2 results in the automatic generation of Restart files at specific intervals of time.
 - d. The arguments of the `RPTCPL` keyword can also be used to control when RESTART files are created.

2. A restart file can also be generated at time 0.0 days, containing the initial solution, either by setting the mnemonic `RESTART` in the `RPTSOL` keyword to 2, or by setting the mnemonic `BASIC` to 2 in keyword `RPTRST` in the `SOLUTION` section. There must be at least one `TIME`, `TSTEP` or `DATES` keyword present to generate the time zero restart file.
3. A `SAVE` file, containing the non-recurrent data of the base run, is generated by specifying the `SAVE` keyword in the `RUNSPEC` section. A `SAVE` file is required if a fast restart is to be performed.
4. If you want the time-dependent restart data to be appended to the `SAVE` file, use the `SAVE` mnemonic in keyword `RPTRST`. This controls the frequency with which the restart data is written to the `SAVE` file, in an analogous manner to the `BASIC` mnemonic. The `SAVE` keyword must also be present in the `RUNSPEC` section. If the `RESTART` file is to be used for graphics output only, unwanted restart arrays may be prevented from being written to this file by using the `NORST` mnemonic (switch 16) in the `RPTRST` keyword.

Eclipse 300

In Eclipse 300, use keyword `RPTRST` to control the output and contents of the `RESTART` file.

SCHEDULE section data not stored on the RESTART file

Apart from the exceptions detailed below, all the Schedule section data from the base run is written to the `RESTART` file and applied in the restart run. The exceptions are the following keywords, whose data is not carried over on the `RESTART` file:

- VFP tables (keywords `VFPPROD`, `VFPINJ`)
- Reporting instructions (keywords `RPTSCHED`, `RPTRST`)
- The “global keywords” that can be entered in any section (`COLUMNS`, `DEBUG`, `ECHO`, `EXTRAPMS`, `FORMFEED`, `INCLUDE`, `MESSAGES`, `NOECHO`, `NOWARN`, `OPTIONS`, `OPTIONS3`)
- Annual scheduling files in the Eclipse gas field operations model extension for Eclipse 100 (keywords `GASBEGIN`, `GASEND` and any keywords in between)
- The keywords `SLAVES`, `DUMPCUPL` and `USECUPL` in the Eclipse Reservoir Coupling Facility extension for Eclipse 100.
- Well PI scaling tables (keyword `PIMULTAB`).
- Scale deposition and damage tables (keywords `SCDPTAB` and `SCDATAB`).
- The inclination factor table for the drift flux model in multisegment wells (keyword `WSEGDFIN`).
- Gas plant tables (keywords `GPTABLE`, `GPTABLEN` and `GPTABLE3`)
- Information associated with the `AIMS` keyword is not transferred across restarts.
- The `TUNING` keyword.
- Non-neighbor connection transmissibility. This is stored in the `INIT` file, but changes made to it in the schedule section, for instance using the `MULTX` keyword, are not carried over for restarts. Any restart for a run which changes non-neighbor transmissibilities in the schedule section must be made before the transmissibilities are changed.

Eclipse 300

Note: These keywords must be present in the `SCHEDULE` section of the restart run, if they are needed in the run.

Notes

The following data cannot be changed at a restart:

- The grid dimensions `NDIVIX`, `NDIVIY`, `NDIVIZ`, and the number of active cells.
- The radial/Cartesian switch.
- The phases present and the permitted mass transfers
- (OIL, WATER, GAS, dissolved GAS, vaporized OIL).
- The number of components present.
- The unit convention.
- The vertical equilibrium options, keyword `VE` in `RUNSPEC`.
- The type of well model, for existing wells.
- Passive tracers can be added in a flexible restart, even if the base run had no tracers. They should be initialized as usual in the `SOLUTION` section with keyword `TBLK` or `TVDP`.
- The maximum number of wells, groups, connections per well and wells per group may be increased at a flexible restart (keyword `WELLDIMS` in the `RUNSPEC` section), but cannot be decreased.
- A warning message is issued if the `RESTART` file was not generated by the current version of Eclipse. In general, using flexible restarts, Eclipse can usually be restarted from restart files written by the two previous versions of the program, unless options not available in the original run are to be used in the restarted run. A restart file produced by Eclipse cannot be loaded into a previous version of the code. If it is not possible to restart the run from the Restart file, an error message is issued.
- A `RESTART` file from a parallel run will have some structural differences from one generated from a serial run using the same data. In particular, some connections will be represented by non-neighbor connections in parallel and neighbor connections in serial. This means that, when visualizing results from a parallel run, there will be planes of cells where neighbor transmissibility appears to be zero at boundaries between processors, and also (for Eclipse 300 using the WARP solver) at boundaries between sub-grids.

Eclipse 100

Eclipse 100

Eclipse 300

Units

x	Eclipse 100
x	Eclipse 300

There are four unit conventions currently used:

- METRIC units
- FIELD units
- LAB units
- PVT-Metric units (in Eclipse 300)

The LAB convention is intended for use when simulating laboratory-scale experiments. It is based on the original Darcy units, except that the unit of time is the hour rather than the second. The units for each data quantity are shown below.

Quantity	Metric	Field	Lab	PVT-M
Length, depth, radius	m	ft	cm	m
Time	day	day	hr	day
Area	m ²	ft ²	cm ²	m ²
Density	kg/m ³	lb/ft ³	gm/cc	kg/m ³
Pressure (absolute)	barsa	psia	atma	atma
Pressure (difference)	bars	psi	atm	atm
Temperature (absolute)	K	°R	K	K
Temperature (relative and difference)	°C	°F	°C	°C
Compressibility	1/bars	1/psi	1/atm	1/atm
Viscosity	cP	cP	cP	cP
Permeability	mD	mD	mD	mD
Liquid surface volume	sm ³	stb	scc	sm ³
Gas surface volume	sm ³	Mscf	scc	sm ³
Reservoir volume	rm ³	rb	rcc	rm ³
Volume	m ³	ft ³	cc	m ³
Liquid surface volume rate	sm ³ /day	stb/day	scc/hr	sm ³ /day
Gas surface volume rate	sm ³ /day	Mscf/day	scc/hr	sm ³ /day
Reservoir volume rate	rm ³ /day	rb/day	rcc/hr	rm ³ /day
Formation volume factor (liquid)	rm ³ /sm ³	rb/stb	rcc/scc	rm ³ /sm ³
Formation volume factor (gas)	rm ³ /sm ³	rb/Mscf	rcc/scc	rm ³ /sm ³
Gas-oil ratio	sm ³ /sm ³	Mscf/stb	scc/scc	sm ³ /sm ³
Oil-gas ratio	sm ³ /sm ³	stb/Mscf	scc/scc	sm ³ /sm ³
Transmissibility	cP.rm ³ /day/bars	cP.rb/day/psi	cP.rcc/hr/atm	cP.rm ³ /day/atm

Quantity	Metric	Field	Lab	PVT-M
Mass	kg	lb	gm	kg
Moles	kg-M	lb-M	gm-M	kg-M
Energy	kJ	Btu	J	kJ
Calorific value	kJ/sm ³	Btu/Mscf	J/scc	kJ/sm ³
Heat capacity	kJ/kg/K	Btu/lb/°R	J/gm/K	kJ/kg/K
Volumetric heat capacity	kJ/m ³ /K	Btu/ft ³ /°R	J/cc/K	kJ/m ³ /K
Thermal conduction transmissibility	kJ/day/K	Btu/day/°R	J/hr/K	kJ/day/K
Thermal conductivity	kJ/m/day/K	Btu/ft/day/°R	J/cm/hr/K	kJ/m/day/K
Surface tension	dynes/cm	dynes/cm	dynes/cm	dynes/cm
Surface tension (E100 foam and surfactant models)	N/m	lbf/in	dynes/cm	N/m
Velocity	m/day	ft/day	cm/hr	m/day
Velocity (Multisegment Well Model)	m/s	ft/s	cm/s	m/s
Device strength	bars.day ² /(kg.m ³)	psi.day ² /(lb.ft ³)	atm.hr ² /(gm.cc)	atm.day ² /(kg.m ³)

Table 11.7: A table of the units used for four conventions

Note: The standard definition of 1 mole of an element or compound is in gram-mole and is in terms of the number of atoms or molecules. There are $6.022 \cdot 10^{23}$ atoms in 1 mole of any element. Everything is relative to Carbon-12 so that 1 mole of Carbon-12 weighs 12 grams. In order to work in different unit systems, the number of moles is reported in kilogram-mole (“kg-M”), pound-mole (“lb-M”) or gram-mole (“gm-M”). They are defined for Carbon-12 so that 1 kg-M weighs 12 kg and 1 lb-M weighs 12 lb. This means that there are 1000 times more atoms in 1 kg-M than in 1 gm-M. The relationships with the standard mole are: 1 kg-M = 10^3 mol, 1 lb-M = $0.45359237 \cdot 10^3$ mol, 1 gm-M = 1 mol.

Note: The multiplier M (used for the volume outputs) stands for a million in all units except for the FIELD units, where it means a thousand.

Constants

The following table gives the values of some principal constants in the four unit conventions.

Quantity	Metric	Field	Lab	PVT-M
Gravity constant (E100)	0.0000981 m ² bars/kg	0.00694 ft ² psi/lb	0.000968 cm ² atm/gm	
Gravity constant (E300)	0.0000980665 m ² bars/kg	0.00694444 ft ² psi/lb	0.000967841 cm ² atm/gm	0.0000967841 m ² atm/kg
Darcy constant (E100)	0.008527	0.001127	3.6	

Quantity	Metric	Field	Lab	PVT-M
Darcy constant (E300)	0.00852702	0.00112712	3.6	0.00864
Atmospheric pressure (E100)	1.013 barsa	14.7 psia	1.0 atma	
Atmospheric pressure (E300)	1.01325 barsa	14.6959 psia	1.0 atma	1.0 atma
Density of air (E100)	1.22 kg/m ³	0.0763 lb/ft ³	0.00122 gm/cc	
Density of air (E300)	1.2232 kg/m ³	0.076362 lb/ft ³	0.0012232 gm/cc	1.2232 kg/m ³
Density of water (E100)	1000.0 kg/m ³	62.4 lb/ft ³	1.0 gm/cc	
Density of water (E300)	999.014 kg/m ³	62.3664 lb/ft ³	0.999014 gm/cc	999.014 kg/m ³
Gas constant, R	0.083143 m ³ bars/K/kg-M	10.732 ft ³ psi/°R/lb-M	82.055776 cc.atm/K/gm-M	0.08205576 m ³ atm/K/kg-M

Table 11.8: Constants used in the four unit conventions

Standard conditions are taken as one atmosphere and 60 °F.

The Darcy constant is defined by:

- 1 mD.m = 0.00852702 cP.rm³/day/bars
- 1 mD.ft = 0.00112712 cP.rb/day/psi
- 1 mD.cm = 3.6 cP.rcc/hr/atm
- 1 mD.m = 0.00864 cP.rm³/day/atm

Conversion factors

The following table gives some useful conversion factors between the unit systems.

Quantity	Metric	Field	Other units
Length	1 m	=3.28084 ft	
	0.3048 m	=1 ft	
Volume	1 m ³	=35.31466 ft ³	=6.289811 bbl
	0.02831685 m ³	=1 ft ³	=0.1781076 bbl
Mass	1 kg	= 2.204623 lb	=0.001 (metric) ton
	0.45359237 kg	=1 lb	=0.0005 (short) ton
Density	1 kg/m ³	=0.06242797 lb/ft ³	=10 ⁻³ gm/cc
	16.01846 kg/m ³	=1 lb/ft ³	=0.01601846 gm/cc

Quantity	Metric	Field	Other units
Pressure	1 bars	=14.50377 psi	=0.986923 atm
	0.06894757 bars	=1 psi	=0.068046 atm
Gas-liquid ratio	1 m ³ /m ³	=5.614583·10 ⁻³ Mcf/bbl	
	178.1076 m ³ /m ³	=1 Mcf/bbl	
Temperatures	1 K	=1/1.8 °R	

Table 11.9: Some useful conversion factors

A note regarding the use of the **ton** as a unit of mass:

- In metric-based systems, a ton (also called a **metric ton** or **tonne**) is 1,000 kg.
- In imperial-based systems, a **short ton** is 2,000 lbs (this is also known as a **net ton** and is the common definition of a **ton** in the U.S.) while a **long ton** is 2,240 lbs
- Prior to 2007.1, when working in **FIELD** units, Eclipse 300 reported both mass rates and mass totals (for example, the **WOMR** and **GWMT** summary keywords) in **long tons**. This was changed to **short tons** at 2007.1. The previous behavior can be recovered by setting [item 112](#) of **OPTIONS3** to a non-zero value. An 'L' (for long) or 'S' (for short) has been added to the name of the unit in the output from 2007.1 to clarify which type of ton is being used.

API gravity

API gravity = (141.5 / liquid gravity) - 131.5

where the liquid gravity is calculated as the liquid oil density divided by the water density at standard conditions.

Conversion for an ideal gas (Z=1)

For an 'ideal gas' Z-factor of unity, we can use the ideal gas equation:

$$PV = nRT \quad \text{Eq. 11.3}$$

where:

P is pressure

V is volume

n is number of moles

R is the gas constant

T is temperature

Using equation [11.3](#), we can calculate the following:

- When $P = 14.7$ psia and $T = 519.67$ °R,
 - the volume occupied by one lb-mole of gas is:

$$V_{\text{molar}} = 379.39445 \text{ ft}^3$$

- the number of lb-moles in unit volume is:

$$n_{\text{moles}} = 0.00263578 \text{ lb-M/ft}^3$$

- When $P = 1.013$ barsa and $T = 288.15$ K,

- the volume occupied by one kg-mole of gas is:

$$V_{\text{molar}} = 23.650203 \text{ m}^3$$

- the number of kg-moles in unit volume is:

$$n_{\text{moles}} = 0.04228293 \text{ kg-M/m}^3$$

Compatibility between Eclipse 100 specialist extensions

x	Eclipse 100
	Eclipse 300

Eclipse 100 contains a collection of specialist extensions to the simulator. Many of these extensions are normally used alone, without the need to apply other extensions in the same run. However, it is often desirable to combine two or more specialist extensions in a single run, for example using the gas field operations model with networks. Some combinations of options are naturally incompatible (for example, coal bed methane and the foam model), while other combinations do not yet work together because their compatibility has been assigned a lower priority for development.

This chapter summarizes the combinations of Eclipse 100 specialist extensions that are currently incompatible. For a more detailed account of the restrictions on the use of each option, please refer to the appropriate chapter in this manual.

Compatibility information

Coal bed methane model

The coal bed methane model is incompatible with:

- The foam model.
- The surfactant model.

Environmental tracers

Environmental tracers are incompatible with:

- Local grid refinement (if the tracer is diffusing).

Flux boundary conditions

When using flow boundary conditions, the FLUX file contains oil/water/gas flows, together with tracer, brine and surfactant flows. Thus it is incompatible with any other option including another fluid or conservation equation. For this reason some combinations are banned. However, the option may still be useful in cases where the other fluid is contained within the [USEFLUX](#) region. If this is the case, the ban may be lifted using [item 232](#) in `OPTIONS`. When pressure boundary conditions are used, the FLUX file also contains solution data for tracers, API, brine tracking, polymer, foam and surfactant.

Flux boundary conditions are incompatible with:

- The gas field operations model.
- Gas lift optimization.
- The Gi pseudo-compositional model.

Foam model

The foam model is incompatible with:

- The coal bed methane model.

It is also not fully integrated with the surfactant model.

Gas calorific value control

There are limitations when using reservoir coupling

Gas field operations model

The gas field operations model is incompatible with:

- Flux boundary conditions.
- Parallel options.

There are also limitations when using reservoir coupling.

Gas lift optimization

Gas lift optimization is incompatible with:

- Flux boundary conditions.
- Reservoir coupling.

Gi pseudo-compositional model

The Gi pseudo-compositional model is incompatible with:

- Flux boundary conditions.
- Local grid refinement.
- Multisegment wells.
- The wellbore friction option.

Gradient option

The gradient option is incompatible with:

- Reservoir coupling.

When run with local grid refinement, only in-place refinements are allowed.

Local grid refinement and coarsening

The local grid refinement and coarsening option is incompatible with:

- Environmental tracers (if the tracer is diffusing).
- The Gi pseudo-compositional model.

In addition, the auto refinement option cannot be run with:

- Multisegment wells.
- The wellbore friction option.

Multisegment wells

Multisegment wells are incompatible with:

- The Gi pseudo-compositional model.
- The auto refinement option in local grid refinement.
- The solvent model.
- The wellbore friction option.

Note also that multisegment wells cannot be used with the temperature option in Eclipse 100.

Network option

The network option has no restrictions on its use with other options.

Parallel options

Parallel options cannot be used with:

- The gas field operations model.
- The **VISCD** keyword.

Reservoir coupling

Reservoir coupling is incompatible with:

- Gas lift optimization.
- The gradient option.

It also has limited capabilities with:

- Gas calorific value control.
- The gas field operations model.

Solvent model

The solvent model is incompatible with:

- Multisegment wells.
- the wellbore friction option.

Note also that wells are not allowed to crossflow when this model is in use.

Surfactant model

The surfactant model is incompatible with:

- Relative permeability hysteresis in the oil and water phases.
- The coal bed methane model.

Wellbore friction option

The wellbore friction option is incompatible with:

- The Gi pseudo-compositional model.
- The auto refinement option in local grid refinement.
- Multisegment wells.
- The Solvent model.

Note also that the Wellbore friction option cannot be used with the miscible flood and brine tracking options in Eclipse 100.